

Chapitre 5 – Android - Bases de données distantes avec Volley

Sommaire

I.	Préparer le projet.....	1
II.	Echange de données en texte brut.....	1
III.	Echange de données en JSON.....	2

I. Préparer le projet

1. Ajouter dans build.gradle (module) la librairie :

```
implementation("com.android.volley:volley:1.2.1")
```

Mettre en version "catalogue" (menu contextuel) :

```
implementation(libs.volley)
```

Synchroniser le projet (importer réellement la bibliothèque)

2. Ajouter la permission *Internet* dans le manifest :

```
<uses-permission android:name="android.permission.INTERNET" />
```

3. Permettre la communication en clair (http) vers le serveur exceptionnellement

Modifier le *manifest* (nœud *application*) :

```
android:networkSecurityConfig="@xml/network_security_config"  
android:usesCleartextTraffic="true"
```

Créer le fichier de ressources *xml/network_security_config* :

```
<?xml version="1.0" encoding="utf-8"?>  
<network-security-config>  
  <domain-config cleartextTrafficPermitted="true">  
    <domain includeSubdomains="true">192.168.0.27</domain>  
  </domain-config>  
</network-security-config>
```

II. Echange de données en texte brut

Construire l'interface (*activity_main.xml*)

1. Java : Appel de la méthode pour *appel du service web* :

```
getServerDataTexteBrut (urlServiceWeb);
```

2. Java : Définition de la méthode pour *appel du service* :

```
private void getServerDataTexteBrut (String url) {  
    RequestQueue requestQueue = Volley.newRequestQueue(this);  
    StringRequest stringRequest  
        = new StringRequest(  
        Request.Method.GET, url,
```



```

        new Response.Listener<String>() {
            @Override
            public void onResponse(String response) {
                tvResultat.append(response);
            }
        },
        new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error)
            {
                tvResultat.append("Erreur : " + error.getLocalizedMessage());
            }
        });
requestQueue.add(stringRequest);
}

```

Explications :

- *RequestQueue* sert à soumettre les requêtes. D'autres manières d'initialiser existent (ici, l'objet est créé à chaque fois)
- *StringRequest* sert à récupérer une chaîne. Il existe aussi *JsonObjectRequest*, *JSONArrayRequest* et *ImageRequest*
- Après GET et l'url du service, vient la méthode appelée en cas de succès (observer dans la signature le *String*, du fait de *StringRequest*), puis la méthode en cas d'échec (erreur réseau, ...)
- Après construction de la requête, elle est ajoutée à la queue (fonctionnement asynchrone - thread)

3. Mise en forme "lambda" des méthodes (moins verbeux) :

```

private void getServerDataTexteBrut (String url) {
    RequestQueue requestQueue = Volley.newRequestQueue(this);
    StringRequest stringRequest
        = new StringRequest(
            Request.Method.GET, url,
            response -> tvResultat.append(response),
            error -> tvResultat.append("Erreur : " + error.getLocalizedMessage()));
    requestQueue.add(stringRequest);
}

```

Explications :

Seuls les noms de paramètres restent, la classe utilisée et la méthode surchargée est déduite

III. Echange de données en JSON

Construire l'interface (*activity_main.xml*)

1. Java : Appel de la méthode *appel du service* :

```
getServerDataJSON (urlServiceWeb);
```

2. Java : Définition de la méthode *appel du service* :

```

private void getServerDataJSON (String url) {
    RequestQueue requestQueue = Volley.newRequestQueue(this);
    JSONArrayRequest jsonArrayRequest = new JSONArrayRequest(
        Request.Method.GET, url,
        null,
        new Response.Listener<JSONArray>() {
            @Override
            public void onResponse(JSONArray response) {

```



```

        try {
            for (int i = 0; i < response.length(); i++) {
                JSONObject jsonObject = response.getJSONObject(i);
                String ville = jsonObject.getString("nom");
                String cp = jsonObject.getString("cp");
                tvResultat.append(ville + " (" + cp + ")\n");
            }
        } catch (JSONException e) {
            tvResultat.append("Erreur JSON : " + e.getLocalizedMessage());
        }
    },
    new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            tvResultat.append("Erreur : " + error.getLocalizedMessage());
        }
    });
requestQueue.add(jsonArrayRequest);
}

```

Explications :

- *JSONArrayRequest* est utilisé puisque le service renvoie un tableau d'objets en JSON
- L'objet *JSONArray* est parcouru, et chaque objet est décodé
- Le reste est similaire au cas précédent

3. Mise en forme "lambda" des méthodes (moins verbeux) :

```

private void getServerDataJSON (String url) {
    RequestQueue requestQueue = Volley.newRequestQueue(this);
    JSONArrayRequest jsonArrayRequest = new JSONArrayRequest(
        Request.Method.GET, url,
        null,
        response -> {
            try {
                for (int i = 0; i < response.length(); i++) {
                    JSONObject jsonObject = response.getJSONObject(i);
                    String ville = jsonObject.getString("nom");
                    String cp = jsonObject.getString("cp");
                    tvResultat.append(ville + " (" + cp + ")\n");
                }
            } catch (JSONException e) {
                tvResultat.append("Erreur JSON : " + e.getLocalizedMessage());
            }
        },
        error -> tvResultat.append("Erreur : " + error.getLocalizedMessage()));
    requestQueue.add(jsonArrayRequest);
}

```

Remarque :

Le respect de l'indentation est primordial pour la lisibilité ...